

操作系统研讨课

Course: B0911011Y

卢天越

2026 春季学期



Project 3 Interactive OS and Process Management

2026.4.10



今日课程内容

- Project 3介绍
- Project 2验收
 - 直接在开发板上验收



截止P2

- 实现了进程调度，具备了例外处理能力
- 用户程序分时复用处理器资源，从视觉上呈现多个进程同时运行的效果
- P3：继续丰富操作系统功能
 - ① 实现一个操作系统终端
 - ② 实现进程间通信
 - ③ 启动双核



操作系统与用户的交互

- 在P1的时候：通过输入数字或程序名称来选择启动程序；用户的输入有回显
- P2为什么没有了：没有一个用户程序会接收用户输入并处理

```
Last login: Sat Mar 29 06:30:01 2025  
stu@stu:~$ █
```

```
C:\Windows\system32\cmd.exe  
Microsoft Windows [版本 10.0.19045.5608]  
(c) Microsoft Corporation. 保留所有权利。  
  
C:\Users\lutia> █
```

- shell：一个可以解析用户输入指令的进程
 - 1、从输入流接收用户的输入，并回显到屏幕上
 - 2、对于输入的字符串进行分析处理
 - 3、执行对应的命令



任务1：终端

- 1、从输入流接收用户的输入，并回显到屏幕上
 - 这个功能在P1的时候已经做过，很熟悉了
- 2、对于输入的字符串进行分析处理
 - 第一步接收到的字符串以回车为结尾
 - 由于命令的复杂性，需要用空格进行分割，分割出第一个字符串为命令，后面是参数
- 3、执行对应的命令
 - 本任务要求实现的命令：ps、clear、exec、kill



shell的说明

- 屏幕下半为shell，上半为其他用户程序的输出
- 建议实现滚屏，否则就要频繁clear
 - 超出屏幕显示范围时依然可以输入，只是输出不到屏幕上
- 要求支持退格
- 要求对错误输入的命令显示
unknown command
- ps要求打印进程信息：
 - PID、状态
- kill的参数为PID
- clear只清下半屏幕

```
> [TASK] I want to wait task (pid=3) to exit.  
> [TASK] I am task with pid 3, I have acquired two mutex locks. (4737)  
> [TASK] I want to acquire mutex lock1 (handle=0).
```

```
----- COMMAND -----  
> root@UCAS_OS: exec waitpid &  
Info: execute waitpid successfully, pid = 2 ...  
> root@UCAS_OS: ps  
[Process Table]:  
[0] PID : 1  STATUS : RUNNING  
[1] PID : 2  STATUS : BLOCKED  
[2] PID : 3  STATUS : READY  
[3] PID : 4  STATUS : BLOCKED  
> root@UCAS_OS: ss  
Error: Unknown Command ss!  
> root@UCAS_OS:
```



exec和kill：进程的启动和杀死

- exec带参数启动一个进程
 - S-core后面带一个id即可，A-core后面需要带名字
- 难点：进程启动也需要带参数
 - S-core：参数直接传进寄存器即可
 - 寄存器数量有限，缺乏扩展性
 - A-core：argc/argv，拷贝进用户栈里
 - argc代表参数的个数
 - argv可以认为是一个指针数组，数组个数为argc



截止任务1

- 程序已经可以在shell中，通过exec的方式运行
- P2的这些程序如果加入编译，也可以这样运行
 - test_project3文件夹中依然有fly.c
- 下一步：让多个进程之间配合运行
 - 不再只是各跑各的
 - 同步、通信：synchronization、communication



任务2：同步原语

- 通过P2中实现的锁，已经可以实现多个进程的同步功能
- 同步原语的核心功能依然是确保同一时间只有一个进程可以访问临界区资源
 - 适用于不同场景
- 本实验要求实现的同步原语：barrier、condition variables
- 任务2还包括一个mailbox的实现
- S-core只需完成任务2的barrier



barrier测试

- test/test_project3/test_barrier.c
- 本测试s-core必须完成
- 三个程序都进入barrier之后才会继续运行
 - 测试时可能会杀死一个
 - ps能看到有进程阻塞

```
> [TASK] Exited barrier (3).(sleep 3 s)
> [TASK] Exited barrier (3).(sleep 2 s)
> [TASK] Exited barrier (3).(sleep 3 s)
```

```
----- COMMAND -----
> root@UCAS_OS: exec barrier &
Info: execute barrier successfully, pid = 2 ...
> root@UCAS_OS:
```



Condition_variables测试

- test/test_project3/test_condition.c
- 一个生产者
- 三个消费者
 - 测试时可能杀死一个
 - ps能看到进程阻塞

```
> [TASK] Total produced 12 products. (next in 4 seconds)
> [TASK] Total consumed 3 products. (Sleep 2 seconds)
> [TASK] Total consumed 3 products. (Sleep 2 seconds)
> [TASK] Total consumed 3 products. (Sleep 2 seconds)
```

```
----- COMMAND -----
> root@UCAS_OS: exec condition &
Info: execute condition successfully, pid = 2 ...
> root@UCAS_OS:
```



mailbox

- mailbox通信：
 - 临界区：发送方发送至信箱，接收方从信箱中取出
- 互斥：发送和接收只能同时进行一个
- 如果发送时信箱满了怎么办？
 - 发送方进程阻塞，在信箱不满的时候解除阻塞
 - 需要考虑剩余空间与发送内容大小的关系
- 接收时信箱为空
 - 类似的阻塞接收方进程



截止任务2

- 到任务2为止，我们实现了shell以及进程间通信
- 这给多核运行提供了条件，多个核同时运行进程并不会冲突
 - 双核有这明显的优势，但也给操作系统带来了挑战
 - 如何启动双核？
 - 双核的例外处理采用一份代码？
 - 进程如何在双核上顺利的调度？
- 在板卡上实现双核运行程序，是任务3的主要内容
 - 把双核启动起来，比在双核上运行程序要难



任务3：双核运行

- 使用loadbootm启动之后，两个核就都会运行
 - 都会跳转到BootLoader运行
 - 实际上存在临界区竞争问题
- 解决办法：在BootLoader里让从核先停下来，等待主核做完初始化之后唤醒从核



- 难点：
 - 1、谁是主核谁是从核
 - 2、从核怎么如何停下来，如何唤醒，何时唤醒



双核运行的难点

- 1、谁是主核谁是从核
 - 通过mhartid这个CSR寄存器读出
 - C语言可以调用get_current_cpu_id这个函数（smp.S文件里）
 - 为了简化，bootloader启动时已经读到a0
 - bootblock.S增加：

```
18          // a0 is mhartid
19          bnez a0, secondary
```
- 2、从核怎么如何停下来，如何唤醒，何时唤醒
 - 如何停下来：死循环
 - 如何唤醒：核间中断，用send_ipi函数来触发
 - 何时唤醒：pcb等不需要重复做的初始化做完后
 - 需要给从核提前设置好stvec寄存器



任务3测试程序

- 双核加法
- 测试程序做两次加法，单双核各一次
 - 测试结果打印加法的用时
 - 单核的时间应在双核的1.5倍以上

```
start compute, from = 0, to = 2500000 Done  
start compute, from = 2500000, to = 5000000 Done
```

```
single core: 2036699 ticks, result = 630  
multi core: 1129776 ticks, result = 630
```



任务4：绑核taskset

- C-core任务：把双核用好
- 实现taskset，把指定进程绑定在指定核上运行
 - test_affinity.c：测试程序打开多个行为相同的进程
 - 竞争激烈的核，每个进程运行都较慢
 - 竞争不激烈的核，其进程运行较快
 - 验收时会在运行过程中动态切换绑核

```
start test cpu affinity, pids = {3, 4, 5, 6, 7, }  
[3] integer test (18/150) access: 1 auipc: 520a0078  
[4] integer test (18/150) access: 5 auipc: 520a0078  
[5] integer test (18/150) access: 6 auipc: 520a0078  
[6] integer test (32/150) access: 1 auipc: 520a0078  
[7] integer test (18/150) access: 6 auipc: 520a0078
```



任务5：细粒度内核锁

- 之前双核为了简化，可以使用大内核锁
 - 即只要进入内核态就加锁
- 这样严重影响双核效率，特别是系统调用比较频繁的时候
- 实现细粒度内核锁，运行三个进程
 - producer、consumer、fly，均系统调用频繁
 - 但是fly所调用的printf和同步原语并不冲突，应使用细粒度锁，让这两种系统调用在双核上可以并行
 - 自行修改测试程序，完成测试，观察启动fly与否时consumer的消费速度下降情况，应下降不超过一半



总结P3任务

- S-core任务：任务1、任务2的barrier
 - 实现shell终端，同步原语只实现barrier
- A-core任务：任务1、2、3
 - 实现多种同步原语，完成双核启动
- C-core任务：任务1、2、3、4、5
 - 在A-core的基础上完成绑核和细粒度内核锁



其他注意事项

- A-core在shell中使用exec执行程序之后，需实现默认waitpid等到对应进程执行结束
 - 但这时shell无法继续输入，需要实现对&符的支持
 - 符合常见shell终端的风格

```
> root@UCAS_OS: exec waitpid &  
Info: execute waitpid successfully, pid = 2 ...  
> root@UCAS_OS: exec barrier &  
Info: execute barrier successfully, pid = 5 ...  
> root@UCAS_OS: exec condition  
Info: execute condition successfully, pid = 9 ...
```

- P2只完成S-core的同学：
 - 在测试程序里加上sys_yield可以执行所有程序
 - 只是shell不能一直等输入，需要一个机制让shell等一段时间输入就退出等待让其他进程运行



P3 Design review

- 介绍你的设计，覆盖以下问题：
 - 在shell中怎么处理字符串
 - Kill一个持有锁的进程时，如何处理
 - Exec启动一个进程时如何传参
 - 如何实现同步原语barrier和condition variables
 - Mailbox的数据结构
 - 双核的使能过程，主核做什么，从核做什么



谢谢!

下节课: 4月17日, P3 design review
Any questions?

